# Assignment 1) NLP Course

A) We are asked to assign multiple and different values to min_df & Item_count and evaluate the results.

For evaluation, Calinski-Harabasz is used which is a metric for clustering which represents the ratio of between-cluster dispersion to within-cluster dispersion. A higher value of this index indicates better clustering quality.

Using the code below, we can assign different values to min_df and item_count and evaluate the resulting clustering performance:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import calinski_harabasz_score

min_df_values = [2, 3, 5, 10]
item_count_values = [100, 200, 500]

for min_df in min_df_values:
   for item_count in item_count_values:

      vectorizer = TfidfVectorizer(strip_accents='unicode', min_df=min_df)
      X = vectorizer.fit_transform(persian_dataset['train']['text'][:item_count])

      km = KMeans(n_clusters=2, init='k-means++', max_iter=100)
      km.fit(X)

      ch_index = calinski_harabasz_score(X.toarray(), km.labels_)

      print(f"min_df={min_df}, item_count={item_count}: Calinski-Harabasz Index
= {ch_index:.2f}")
```

1) min_df = 2, Item_count = 100 , Calinski-Harabasz = 2.83

2) min_df = 2, Item_count = 200 , Calinski-Harabasz = 4.18

3) min_df = 2, Item_count = 500 , Calinski-Harabasz = 7.46

4) min_df = 3, Item_count = 100 , Calinski-Harabasz = 3.94

5) min_df = 3, Item_count = 200 , Calinski-Harabasz = 4.68

6) min_df = 3, Item_count = 500 , Calinski-Harabasz =8.20

7) min_df = 5, Item_count = 100 , Calinski-Harabasz = 6.16

8) min_df = 5, Item_count = 200 , Calinski-Harabasz = 6.82

9) min_df = 5, Item_count = 500 , Calinski-Harabasz = 9.40

10) min_df = 10, Item_count = 100 , Calinski-Harabasz = 10.82

11) min_df = 10, Item_count = 200 , Calinski-Harabasz = 10.88

12) min_df = 10, Item_count = 500 , Calinski-Harabasz = 11.96

Here, we can observe that the Calinski–Harabasz score for experiment 12 is higher than those of all the other experiments. Therefore, we can conclude that min_df = 10 and item_count = 500 provide the best results.

We can also observe that increasing both parameters leads to a higher Calinski–Harabasz score. For example, when min_df remains constant and item_count increases, the Calinski–Harabasz score improves. Similarly, when item_count remains constant and min_df increases, the score also improves.

B) We will do the same thing, this time on “persian_news_dataset” . The code below is written and executed for this part:

```
dataset = load_dataset("saied/persian_news_dataset", split="train",
streaming=True)

min_df_values = [2, 3, 5, 10]
item_count_values = [100, 200, 500]
document_count = 1000

for min_df in min_df_values:
    for item_count in item_count_values:
        print(f"min_df={min_df}, item_count={item_count}:")

        docs = []
```

```
        counter = 0
        skip_counter = 0
        for doc in dataset:
            if len(doc['title']) == 0 or len(doc['category']) == 0:
                skip_counter += 1
            docs.append(doc)
            counter += 1
            if counter == document_count:
                break

        vectorizer = TfidfVectorizer(strip_accents='unicode',
min_df=min_df)
        X = vectorizer.fit_transform([doc['text'] for doc in docs])

        km = KMeans(n_clusters=6, init='k-means++', max_iter=100)
        t0 = time()
        km.fit(X[:item_count])

        ch_index = calinski_harabasz_score(X[:item_count].toarray(),
km.labels_)
        print(f"Calinski-Harabasz Index: {ch_index:.2f}")
```

The results are shown below:

1) min_df = 2, Item_count = 100 , Calinski-Harabasz = 1.50

2) min_df = 2, Item_count = 200 , Calinski-Harabasz = 2.18

3) min_df = 2, Item_count = 500 , Calinski-Harabasz = 4.39

4) min_df = 3, Item_count = 100 , Calinski-Harabasz = 1.72

5) min_df = 3, Item_count = 200 , Calinski-Harabasz = 2.37

6) min_df = 3, Item_count = 500 , Calinski-Harabasz = 4.94

7) min_df = 5, Item_count = 100 , Calinski-Harabasz = 1.68

8) min_df = 5, Item_count = 200 , Calinski-Harabasz = 2.24

9) min_df = 5, Item_count = 500 , Calinski-Harabasz = 5.30

10) min_df = 10, Item_count = 100 , Calinski-Harabasz = 1.70

11) min_df = 10, Item_count = 200 , Calinski-Harabasz = 2.17

12) min_df = 10, Item_count = 500 , Calinski-Harabasz = 5.77

Based on the results of this section, we can observe that experiment 12 achieves a higher Calinski–Harabasz score than all the other experiments. Therefore, we conclude that min_df = 10 and item_count = 500 provide the best clustering performance.

When min_df is kept constant and item_count is increased, the Calinski–Harabasz score clearly improves. Similarly, when item_count remains constant and min_df increases, the score also improves.

C) In this section, we are required to implement tokenization and Part-of-Speech (POS) tagging on the Persian_news dataset, following the example provided in the given code.

To accomplish this, we use the provided code and first download and import Stanza, a Python library for Natural Language Processing (NLP). We then use it to tokenize the text and identify the grammatical role of each word in a sentence.

```
dataset = load_dataset("saied/persian_news_dataset", split="train",
streaming=True)
first_doc = next(iter(dataset))
doc = nlp(first_doc['text'])

print(*[f'word: {word.text+" "}\tpos: {word.pos}' for sent in
doc.sentences for word in sent.words], sep='\n')
```

Using this code, we identify the POS tags of all words in the first document of the dataset. The output can also be viewed through the link provided below. The same process can be applied to all documents in the dataset, and the code works correctly. However, for simplicity, we demonstrate it only on the first document.

Furthermore, using the following code snippet, we can retain only four grammatical categories in each document: verbs, nouns, adjectives, and adverbs, while filtering out all other POS categories and words.

```
dataset = load_dataset("saied/persian_news_dataset", split="train",
streaming=True)
first_doc = next(iter(dataset))
doc = nlp(first_doc['text'])

pos_to_keep = {"NOUN", "VERB", "ADV", "ADJ"}

filtered_words = [(word.text, word.pos) for sent in doc.sentences for word
in sent.words if word.pos in pos_to_keep]

for word, pos in filtered_words:
    print(f'Word: {word}\tPOS: {pos}')
```

We are also required to perform clustering after POS tagging and filtering, using the best parameter values obtained in the previous sections.

Based on the results of the previous section, the best parameter values are:

- min_df = 10
- item_count = 500

Using the following code, we perform clustering after tokenization and vectorization using the K-Means algorithm.

```
!pip install datasets
!pip install stanza
import stanza
from datasets import load_dataset
stanza.download('fa')
nlp = stanza.Pipeline(lang='fa', processors='tokenize')
dataset = load_dataset("saied/persian_news_dataset", split="train",
streaming=True)
tokenized_docs = []
for doc in dataset:
    tokenized_doc = nlp(doc['text'])
    tokenized_text = ' '.join([word.text for sent in
tokenized_doc.sentences for word in sent.words])
    tokenized_docs.append(tokenized_text)
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(strip_accents='unicode', min_df=10)
X = vectorizer.fit_transform(tokenized_docs)
```

```
from sklearn.cluster import KMeans
from time import time

km = KMeans(n_clusters=2, init='k-means++', max_iter=100)
t0 = time()
km.fit(X)
print("Clustering done in %0.3fs" % (time() - t0))
from sklearn.metrics import calinski_harabasz_score

ch_index = calinski_harabasz_score(X.toarray(), km.labels_)
print(f"Calinski-Harabasz Index: {ch_index:.2f}")

import pandas as pd

df_cl = pd.DataFrame({"text": [doc['text'] for doc in dataset], "label":
[doc['label'] for doc in dataset]})
df_cl['cluster'] = km.labels_

clusters = df_cl.groupby('cluster')
for cluster in clusters.groups:
    with open('cluster' + str(cluster) + '.csv', 'w') as f:
        data = clusters.get_group(cluster)[['text']]
        f.write(data.to_csv(index_label='id'))

print("Cluster centroids: \n")
order_centroids = km.cluster_centers_.argsort()[:, ::-1]
terms = vectorizer.get_feature_names_out()

for i in range(2):
    print("Cluster %d:" % i)
    for j in order_centroids[i, :10]:
        print(' %s' % terms[j])
    print('------------')
```

https://colab.research.google.com/drive/1chIzdLZCcRK5EeQyc3IV5WGAc42uWWlQ?usp=sharing

Final changes and the final code are located in this Goolge Colab link.